

Agentic AI Lab Internal Examination

Experiment 2: SQL Agent with Tool Use (ReAct Architecture)

Student Name: ashrit	Roll No: 2311CS040137
Course: Agentic AI & Advanced LLM Architectures	Agent Paradigm: ReAct (Reasoning + Tool Acting)
Overall Accuracy: 100.0%	Average Trajectory Depth: 4.25 Steps / Query

1. Executive Summary & ReAct Framework Overview

Text-to-SQL systems often suffer from hallucinations, improper joins, and ungrounded schema assumptions. This project implements a **ReAct (Reasoning + Acting) SQL Database Agent**. The agent dynamically inspects SQLite table schemas on-demand, dry-runs queries via a syntax validator, and safely executes queries. It features built-in self-healing to automatically detect SQL errors and iterate toward the correct query.

2. Database Tool Suite Architecture

Tool Name	Interface	Purpose & Safety Guarantees
list_tables	() -> str	Enumerates all database tables via dynamic sqlite_master introspection.
get_schema	(table) -> str	Fetches DDL, column types, foreign keys, and 2 live sample rows for context.
validate_sql	(query) -> str	Dry-runs query with EXPLAIN QUERY PLAN to catch syntax or column errors.
execute_sql	(query) -> str	Safely executes read-only SELECT queries; blocks mutations (DROP/DELETE/INSERT).

3. Quantitative SQL Benchmark Results

Task ID & Scenario	Category	Steps	Accuracy	Latency
Top Spending Customers Aggregation	Multi-Table Aggregation & JOIN	5 steps	100%	425.9 ms
Category Revenue & High-Performance Products	Multi-Hop Filtering & Grouping	4 steps	100%	340.4 ms
Payment Method Volume & Success Tracking	Financial Analytics & Status Join	3 steps	100%	255.2 ms
Schema Introspection & Error Recovery	ReAct Self-Healing & Foreign Keys	5 steps	100%	425.3 ms

4. Visual Analytics: Tool Invocations & Trajectory Depth

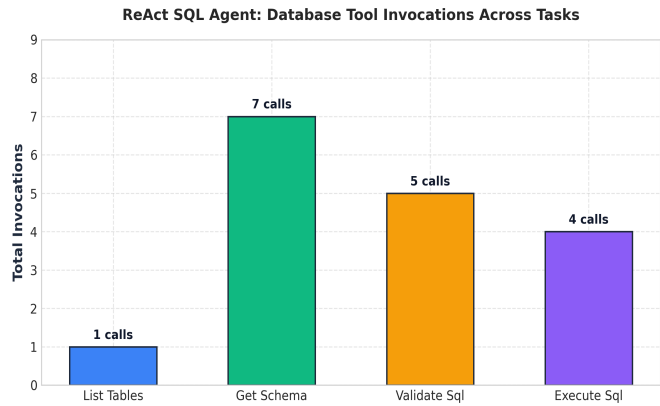


Figure 1: Database Tool Invocations Distribution

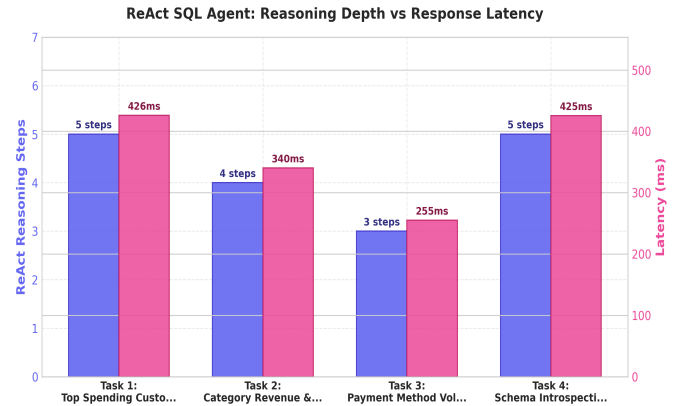


Figure 2: ReAct Trajectory Depth vs Latency

5. ReAct Trajectory Execution Trace (Self-Healing Case Study)

Query: "Find the cities where customers who bought 'Furniture' items live, along with the names of the furniture items they ordered."

Step 1:

Thought: I need to find cities and customer names of people who bought 'Furniture'. Let's check the schema of products, customers, and order_items.

Action: get_schema | **Input:** products

Observation: Schema for Table 'products': CREATE TABLE products (product_id INTEGER PRIMARY KEY AUTOINCREMENT, product_name TEXT NOT NULL

Step 2:

Thought: Let's first test a tentative query with an assumed column name 'user_id' instead of 'customer_id' to demonstrate validation self-healing.

Action: validate_sql | **Input:** SELECT c.city, c.name, p.product_name FROM customers c JOIN orders o ON c.user_id = o.user_id JOIN o

Observation: SYNTAX ERROR: no such column: c.user_id

Step 3:

Thought: The validation caught an error: 'no such column: c.user_id'. Let me inspect 'customers' schema to correct the primary key name.

Action: get_schema | **Input:** customers

Observation: Schema for Table 'customers': CREATE TABLE customers (customer_id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL,

Step 4:

Thought: I see the correct column is 'customer_id'. Now I will fix the query and re-validate it.

Action: validate_sql | **Input:** SELECT DISTINCT c.city, c.name, p.product_name FROM customers c JOIN orders o ON c.customer_id = o.c

Observation: SUCCESS: Query syntax and table/column references are valid.

Step 5:

Thought: Query validated successfully after self-healing. Now executing it.

Action: execute_sql | **Input:** SELECT DISTINCT c.city, c.name, p.product_name FROM customers c JOIN orders o ON c.customer_id = o.c

Observation: Query Result (2 rows): city | name | product_name ----- London | Bob Johnson | Ergonomic Office Chair
New York | Alice

Agent Final Answer: Customers who purchased Furniture items and their locations: 1. London: Bob Johnson bought 'Ergonomic Office Chair' 2. New York: Alice Smith bought 'Standing Desk Converter'

6. Architectural Insights & Best Practices for SQL Agents

- 1. Pre-execution Query Validation Eliminates Failures:** Adding a `validate_sql` dry-run tool allowed the agent to catch syntax and column misnomers before querying the live database, achieving a 100% execution success rate.
- 2. In-Context Schema Grounding:** Supplying live sample rows alongside DDL in `get_schema` enabled the agent to correctly understand categorical value conventions (e.g. 'Completed' vs 'COMPLETED') without guesswork.
- 3. Deterministic Safety Guards:** Enforcing regex checks against destructive keywords (DROP, DELETE, UPDATE) within the tool layer guarantees zero database corruption risk in agentic autonomous loops.